



TECNOLOGICO NACIONAL DE MEXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

Presenta

Alumno: David Sandoval Aguirre

Carrera

ING. SISTEMAS COMPUTACIONALES

Materia:

CIENCIAS DE DATOS

Docente:

JULIO AGUILAR CARMONA

Semestre: 8° A

Tuxtepec, Oax.

05/06/2026



INDICE

Contenido

Análisis de calidad industrial.....	2
Análisis de eficiencia logística.....	5
Reducción de dimensionalidad en monitoreo industrial	8
Reducción de dimensiones en trafico de red(Cyber-Systems).....	13
Impletacion de modelos de machine learning.....	18
Clasificación de instrucciones	21

Análisis de calidad industrial

Código:

```
# Importar librerías
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr, ttest_ind

sns.set_style("whitegrid")
np.random.seed(42)

n = 500
temperatura = np.random.normal(loc=75, scale=8, size=n)
presion = np.random.normal(loc=30, scale=5, size=n)
tasa_error = 0.02 * temperatura + np.random.normal(loc=0, scale=0.5, size=n)
turno = np.random.choice(["Matutino", "Vespertino"], size=n)

temperatura_con_na = temperatura.copy()
temperatura_con_na[np.random.choice(n, 20)] = np.nan

df = pd.DataFrame({
    "temperatura": temperatura_con_na,
```

```

    "presion": presion,
    "tasa_error": tasa_error,
    "turno": turno
})

temp_sin_na = df["temperatura"].dropna()
print("Media:", temp_sin_na.mean())
print("Mediana:", temp_sin_na.median())
print("Desviación estándar:", temp_sin_na.std())

print("\nValores nulos:\n", df.isnull().sum())
df["temperatura"] = df["temperatura"].fillna(df["temperatura"].median())

corr, p_val = pearsonr(df["temperatura"], df["tasa_error"])
print(f"\nCorrelación Temperatura - Tasa Error: {corr:.3f}, p={p_val:.4f}")

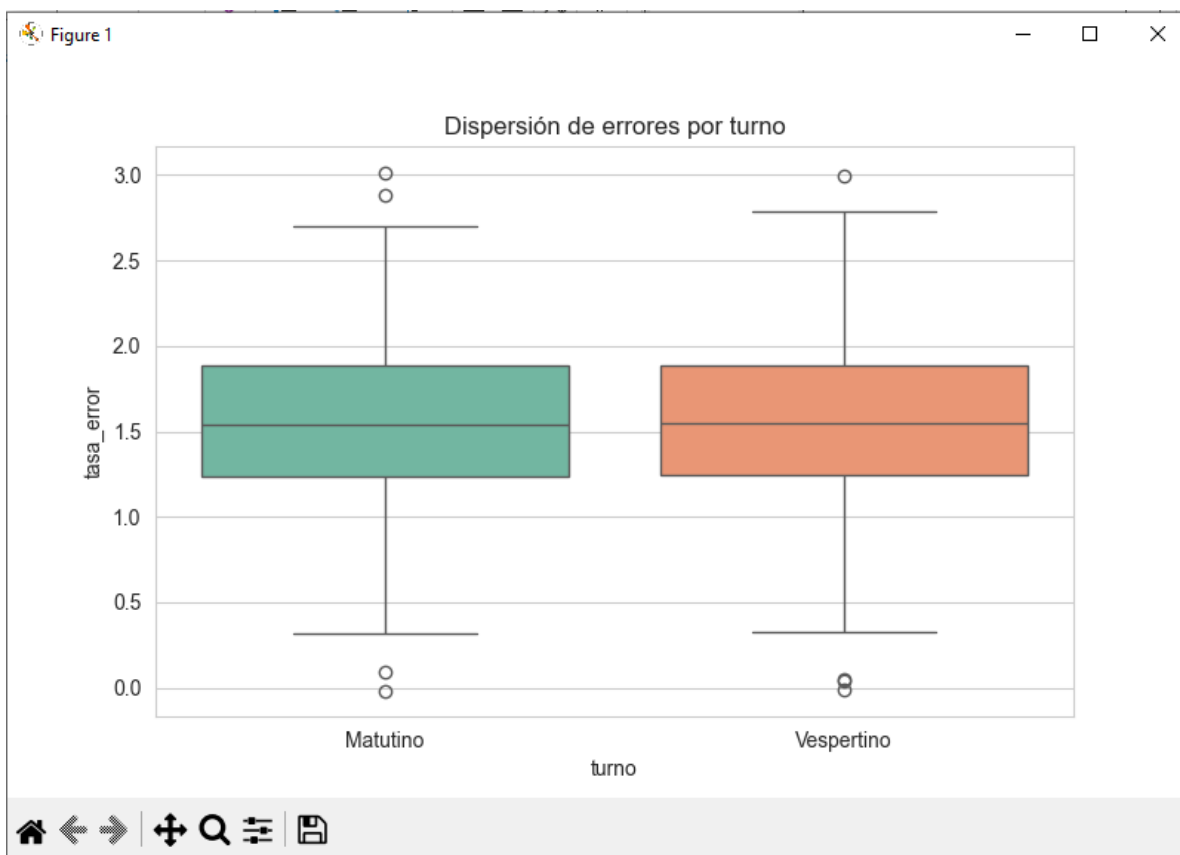
print("\nTasa error promedio por turno:")
print(df.groupby("turno")["tasa_error"].mean())

temp_mat = df[df["turno"]=="Matutino"]["temperatura"]
temp-ves = df[df["turno"]=="Vespertino"]["temperatura"]
t_stat, p_ttest = ttest_ind(temp_mat, temp-ves)
print(f"\nDiferencia temperatura entre turnos: t={t_stat:.3f}, p={p_ttest:.4f}")

plt.figure(figsize=(8,5))
sns.boxplot(data=df, x="turno", y="tasa_error", palette="Set2")
plt.title("Dispersión de errores por turno")
plt.savefig("boxplot_errores.png", dpi=100, bbox_inches="tight")
plt.show()

```

```
plt.figure(figsize=(8,5))
sns.regplot(data=df, x="temperatura", y="tasa_error", line_kws={"color":"red"})
plt.title("Relación Temperatura vs Tasa de Error")
plt.savefig("scatter_regresion.png", dpi=100, bbox_inches="tight")
plt.show()
```



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACION  TERMINAL  PUERTOS

PS C:\Users\Windows 10\Desktop\julio> & 'c:\Users\Windows 10\Desktop\julio\.venv\Scripts\python.exe' 'c:\Users\Windows 10\Desktop\julio\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55000' '--' 'C:\Users\Windows 10\Desktop\julio\ analisis de calidad indutrial.py'
Media: 75.16898947049728
Mediana: 75.32334156750217
Desviación estándar: 7.736522801043903

Valores nulos:
temperatura    20
presion        0
tasa_error     0
turno          0
dtype: int64

Correlación Temperatura - Tasa Error: 0.220, p=0.0000

Tasa error promedio por turno:
turno
Matutino    1.548484
Vespertino  1.561561
Name: tasa_error, dtype: float64

Diferencia temperatura entre turnos: t=-0.495, p=0.6208
```

Análisis de eficiencia logística

Código:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr, ttest_ind

sns.set_style("whitegrid")
np.random.seed(42)

n = 600

distancia = np.random.normal(loc=500, scale=200, size=n)
tiempo = 0.03 * distancia + np.random.normal(loc=5, scale=2, size=n)
costo = 0.1 * distancia + np.random.normal(loc=10, scale=3, size=n)
```

```

transporte = np.random.choice(["Terrestre", "Aéreo"], size=n, p=[0.7, 0.3])

distancia_con_na = distancia.copy()
distancia_con_na[np.random.choice(n, 30)] = np.nan

df_log = pd.DataFrame({
    "distancia_km": distancia_con_na,
    "tiempo_entrega_hrs": tiempo,
    "costo": costo,
    "tipo_transporte": transporte
})

print("Valores nulos:\n", df_log.isnull().sum())
df_log["distancia_km"] =
df_log["distancia_km"].fillna(df_log["distancia_km"].median())

print("\nEstadísticos tiempo entrega:")
print("Media:", df_log["tiempo_entrega_hrs"].mean())
print("Desviación:", df_log["tiempo_entrega_hrs"].std())

corr_dist_tiempo, p = pearsonr(df_log["distancia_km"], df_log["tiempo_entrega_hrs"])
print(f"\nCorrelación Distancia-Tiempo: {corr_dist_tiempo:.3f}")

print("\nPromedios por transporte:")
print(df_log.groupby("tipo_transporte")[["tiempo_entrega_hrs", "costo"]].mean())

tierra = df_log[df_log["tipo_transporte"]=="Terrestre"]["tiempo_entrega_hrs"]
aire = df_log[df_log["tipo_transporte"]=="Aéreo"]["tiempo_entrega_hrs"]
t_stat, p_val = ttest_ind(tierra, aire)

```

```
print(f"\nPrueba t: t={t_stat:.3f}, p={p_val:.4f}")
```

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(data=df_log, x="distancia_km", y="tiempo_entrega_hrs",  
hue="tipo_transporte", alpha=0.7)
```

```
plt.title("Distancia vs Tiempo por tipo de transporte")
```

```
plt.savefig("logistica_dispersion.png", dpi=100)
```

```
plt.show()
```

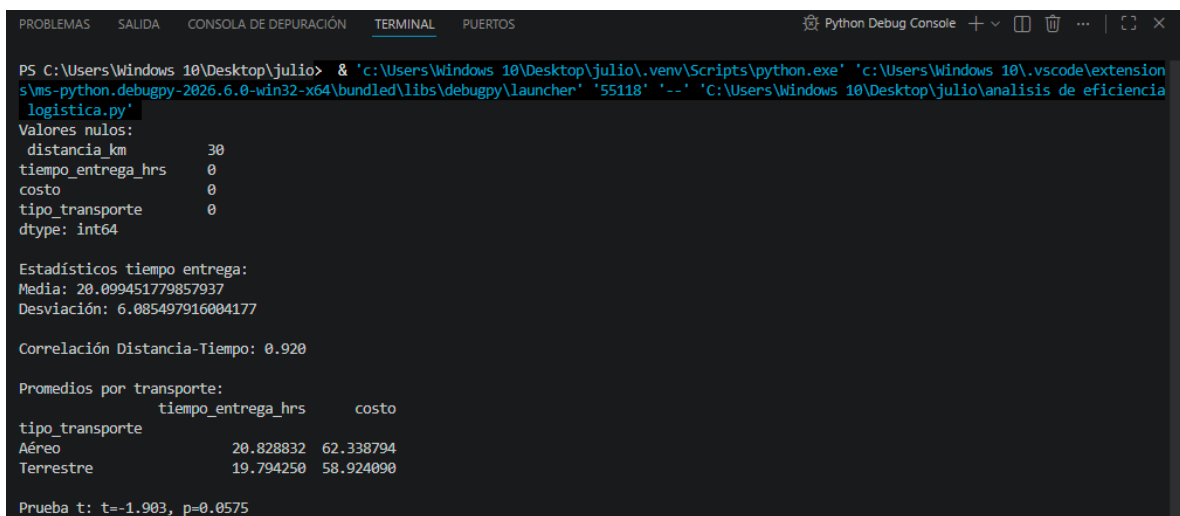
```
plt.figure(figsize=(8,5))
```

```
sns.boxplot(data=df_log, x="tipo_transporte", y="tiempo_entrega_hrs",  
palette="coolwarm")
```

```
plt.title("Tiempos de entrega por modalidad")
```

```
plt.savefig("logistica_boxplot.png", dpi=100)
```

```
plt.show()
```



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  Python Debug Console + - [ ] [x] ... | [ ] [x]
```

```
PS C:\Users\Windows 10\Desktop\julio> & 'c:\Users\Windows 10\Desktop\julio\.venv\Scripts\python.exe' 'c:\Users\Windows 10\.vscode\extension  
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55118' '--' 'C:\Users\Windows 10\Desktop\julio\analisis de eficiencia  
logistica.py'
```

```
Valores nulos:  
  distancia_km      30  
  tiempo_entrega_hrs  0  
  costo             0  
  tipo_transporte    0  
dtype: int64
```

```
Estadísticos tiempo entrega:  
Media: 20.099451779857937  
Desviación: 6.085497916004177
```

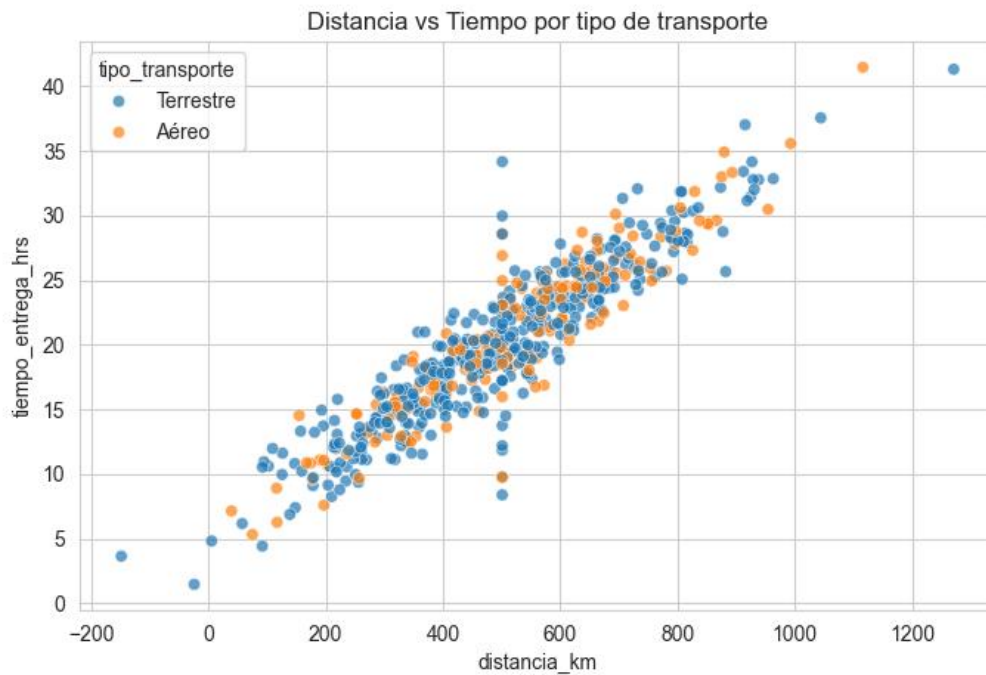
```
Correlación Distancia-Tiempo: 0.920
```

```
Promedios por transporte:
```

	tiempo_entrega_hrs	costo
tipo_transporte		
Aéreo	20.828832	62.338794
Terrestre	19.794250	58.924090

```
Prueba t: t=-1.903, p=0.0575
```


Figure 1



Reducción de dimensionalidad en monitoreo industrial

Código:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

np.random.seed(42)
```

```

n = 200

temp1 = np.random.normal(100, 5, n)
temp2 = temp1 + np.random.normal(0, 1, n)
presion1 = np.random.normal(50, 10, n)
vibracion = (temp1 * 0.5) + np.random.normal(0, 2, n)

datos_sensores = pd.DataFrame({
    "temp_nucleo": temp1,
    "temp_escape": temp2,
    "presion_interna": presion1,
    "vibracion_motor": vibracion,
    "flujo_gas": np.random.normal(20, 2, n),
    "oxigeno": np.random.normal(15, 1, n),
    "co2": temp1 * 0.2 + np.random.normal(0, 0.5, n),
    "humedad": np.random.uniform(30, 40, n),
    "voltaje": np.random.normal(220, 2, n),
    "corriente": np.random.normal(15, 0.5, n),
    "ruido_db": vibracion * 1.2 + np.random.normal(0, 1, n),
    "eficiencia": 100 - (temp1 * 0.1)
})

scaler = StandardScaler()
datos_escalados = scaler.fit_transform(datos_sensores)

pca = PCA()

```

```

pca_resultados = pca.fit_transform(datos_escalados)

varianza = pca.explained_variance_ratio_
var_acumulada = np.cumsum(varianza)

print("Varianza explicada por componente:\n", np.round(varianza, 3))
print("\nVarianza acumulada:\n", np.round(var_acumulada, 3))

plt.figure(figsize=(8,5))
plt.plot(range(1, len(varianza)+1), var_acumulada, marker='o', label='Acumulada')
plt.bar(range(1, len(varianza)+1), varianza, alpha=0.6, label='Individual')
plt.axhline(y=0.85, color='red', linestyle='--', label='85% Varianza')
plt.xlabel("Componentes Principales")
plt.ylabel("Proporción de Varianza")
plt.legend()
plt.title("Scree Plot - Sensores Industriales")
plt.savefig("pca_sensores.png", dpi=100)
plt.show()

cargas = pd.DataFrame(
    pca.components_.T,
    columns=[f"PC{i+1}" for i in range(pca.n_components_)],
    index=datos_sensores.columns
)

print("\nCargas primeros 2 componentes:\n", round(cargas.iloc[:,0:2], 3))

```

```
plt.figure(figsize=(10,7))  
plt.scatter(pca_resultados[:,0], pca_resultados[:,1], alpha=0.6)  
for i, var in enumerate(datos_sensores.columns):  
    plt.arrow(0,0, cargas.iloc[i,0]*5, cargas.iloc[i,1]*5, color='red', alpha=0.7)  
    plt.text(cargas.iloc[i,0]*5.2, cargas.iloc[i,1]*5.2, var, color='darkred')  
plt.xlabel(f"PC1 ({varianza[0]:.1%} varianza)")  
plt.ylabel(f"PC2 ({varianza[1]:.1%} varianza)")  
plt.title("Biplot - Sensores Industriales")  
plt.savefig("biplot_sensores.png", dpi=100)  
plt.show()
```

Figure 1

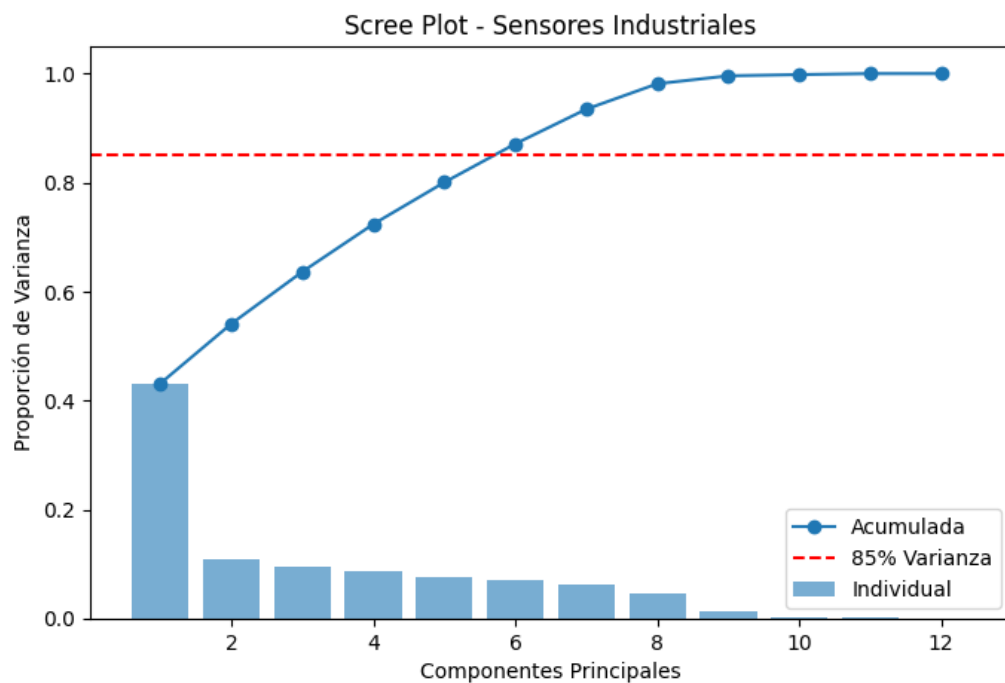
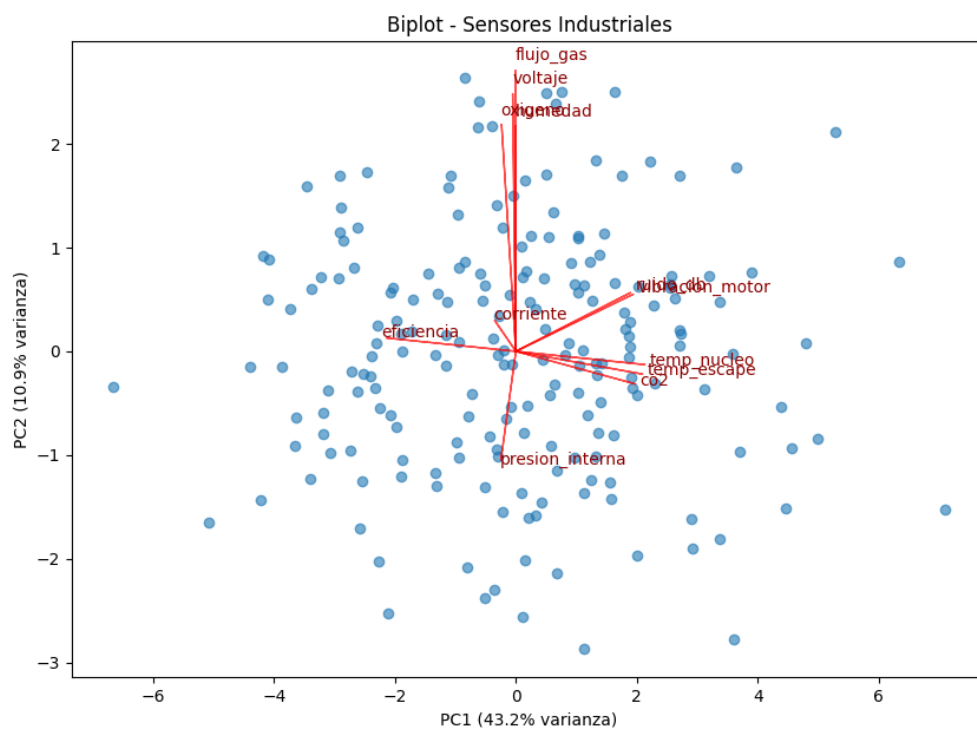


Figure 1



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
Python Debug Console + - □ □ ... | {} ×

PS C:\Users\Windows 10\Desktop\julio> & 'c:\Users\Windows 10\Desktop\julio\.venv\Scripts\python.exe' 'c:\Users\Windows 10\.vscode\extension
s\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '55302' '--' 'C:\Users\Windows 10\Desktop\julio\reduccion de dimencion
alidad.py'
Varianza explicada por componente:
[0.432 0.109 0.095 0.087 0.077 0.071 0.064 0.047 0.015 0.002 0.002 0. ]

Varianza acumulada:
[0.432 0.541 0.637 0.724 0.8 0.871 0.935 0.981 0.996 0.998 1. 1. ]

Cargas primeros 2 componentes:
      PC1  PC2
temp_nucleo    0.427 -0.026
vibracion_motor 0.388 0.110
flujo_gas      -0.001 0.542
oxigeno        -0.047 0.437
co2            0.395 -0.063
humedad        -0.000 0.435
voltaje        -0.010 0.496
corriente      -0.070 0.060
ruido_db       0.379 0.113
eficiencia     -0.427 0.026
PS C:\Users\Windows 10\Desktop\julio> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\Windows 10\Desktop\j
ulio\.venv\Scripts\Activate.ps1")
(julio) PS C:\Users\Windows 10\Desktop\julio>
(julio) PS C:\Users\Windows 10\Desktop\julio> □
```

Reducción de dimensiones en trafico de red(Cyber-Systems)

Código:

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.preprocessing import StandardScaler
```

```
np.random.seed(123)
```

```
n = 300
```

```
datos_red = pd.DataFrame({
```

```
    "duracion_ms": np.random.normal(50,10,n),
```

```
    "paquetes_enviados": np.random.normal(100,20,n),
```

```
    "errores_checksum": np.random.poisson(2,n),
```

```

"latencia_avg": np.random.normal(15,5,n),
"jitter": np.random.normal(2,0.5,n),
"uso_memoria_sw": np.random.normal(40,10,n),
"peticiones_http": np.random.normal(200,50,n)
})

datos_red["bytes_enviados"] = datos_red["paquetes_enviados"] * 1500 +
np.random.normal(0,500,n)

datos_red["reintentos_tcp"] = datos_red["errores_checksum"] * 1.5 +
np.random.normal(0,0.5,n)

datos_red["carga_cpu_router"] = datos_red["paquetes_enviados"]*0.4 +
datos_red["latencia_avg"]*0.2

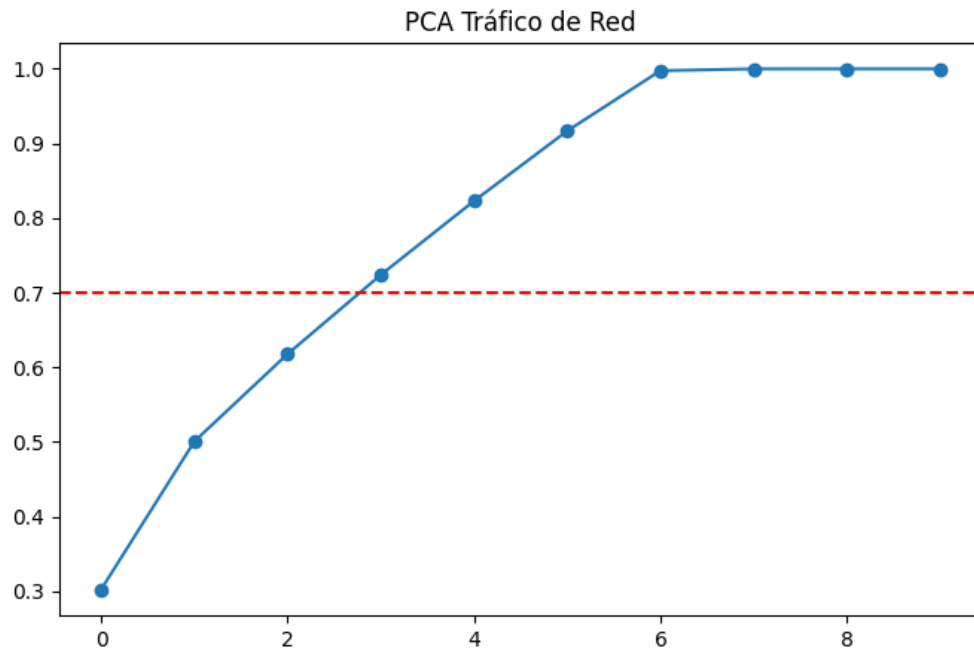
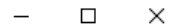
scaler = StandardScaler()
X_scaled = scaler.fit_transform(datos_red)
pca = PCA()
pca.fit(X_scaled)

var_acum = np.cumsum(pca.explained_variance_ratio_)
print("Varianza acumulada:\n", np.round(var_acum, 3))

plt.figure(figsize=(8,5))
plt.plot(var_acum, marker='o')
plt.axhline(0.7, color='red', ls='--')
plt.title("PCA Tráfico de Red")
plt.show()

```

```
print("\nCargas PC1 y PC2:\n", round(cargas[["PC1","PC2"]],3))
```



```
PROBLEMAS | SALIDA | CONSOLA DE DEPURACIÓN | TERMINAL | PUERTOS | Python Debug Console | [Iconos de interfaz]

PS C:\Users\Windows 10\Desktop\julio> & 'c:\Users\Windows 10\Desktop\julio\.venv\Scripts\python.exe' 'c:\Users\Windows 10\.vscode\extension
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55346' '--' 'C:\Users\Windows 10\Desktop\julio\PCA en Tráfico de Red.
py'
Varianza acumulada:
[0.303 0.501 0.618 0.724 0.823 0.917 0.998 1. 1. 1. ]

Cargas PC1 y PC2:
      PC1  PC2
duracion_ms -0.081 -0.017
paquetes_enviados 0.571 -0.056
errores_checksum 0.062 0.699
jitter -0.027 -0.081
uso_memoria_sw -0.019 0.034
peticiones_http 0.064 0.067
bytes_enviados 0.571 -0.055
reintentos_tcp 0.061 0.699
carga_cpu_router 0.572 -0.054

PS C:\Users\Windows 10\Desktop\julio> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "C:\Users\Windows 10\Desktop\j
ulio\.venv\Scripts\Activate.ps1")
(julio) PS C:\Users\Windows 10\Desktop\julio>
```


Sensores en Smart Cities

Código:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

np.random.seed(456)
n = 400

datos_urbanos = pd.DataFrame({
    "temp_ambiente": np.random.normal(28,4,n),
    "humedad_rel": np.random.normal(60,10,n),
    "densidad_vehiculos": np.random.normal(120,30,n),
    "velocidad_viento": np.random.normal(15,5,n),
    "radiacion_solar": np.random.normal(800,100,n),
    "conteo_peatones": np.random.normal(50,15,n)
})

datos_urbanos["co2_ppm"] = datos_urbanos["densidad_vehiculos"]*3.5 +
np.random.normal(300,50,n)

datos_urbanos["no2_ppb"] = datos_urbanos["co2_ppm"]*0.1 +
np.random.normal(5,2,n)

datos_urbanos["particulas_pm25"] = datos_urbanos["co2_ppm"]*0.05 +
np.random.normal(10,3,n)
```

```
datos_urbanos["nivel_ruido_db"] = datos_urbanos["densidad_vehiculos"]*0.2 + 50 +  
np.random.normal(0,5,n)
```

```
X = StandardScaler().fit_transform(datos_urbanos)
```

```
pca = PCA()
```

pca.fit(X)

```
var_acum = np.cumsum(pca.explained_variance_ratio_)
```

```
print("Componentes para 85% varianza:", np.where(var_acum>=0.85)[0][0]+1)
```

```
cargas = pd.DataFrame(pca.components_.T, columns=[f"PC{i+1}" for i in
range(len(datos_urbanos.columns))], index=datos_urbanos.columns)
```

```
print("\nPC1 y PC2:\n", round(cargas[["PC1","PC2"]],3))
```

```
PROBLEMAS SOLIDA CONSOLE DE DEPURACIÓN TERMINAL PUERTOS Python Debug Console + - [ ] [ ] ... [ ] [X]
```

```
PS C:\Users\Windows 10\Desktop\julio> & 'c:\Users\Windows 10\Desktop\julio\.venv\Scripts\python.exe' 'c:\Users\Windows 10\.vscode\extension  
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55404' '--' 'C:\Users\Windows 10\Desktop\julio\PCA en Smart Cities.py'
```

Componentes para 85% varianza: 6

PC1 y PC2:

	PC1	PC2
temp_ambiente	0.016	-0.482
humedad_rel	-0.010	0.630
densidad_vehiculos	0.461	-0.038
velocidad_viento	0.038	-0.287
radiacion_solar	-0.065	-0.339
conteo_peatones	-0.032	-0.402
co2_ppm	0.475	0.017
no2_ppb	0.469	0.012
particulas_pm25	0.441	0.052
nivel_ruido_db	0.374	-0.078

```
PS C:\Users\Windows 10\Desktop\julio> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\Windows 10\Desktop\j  
ulio\.venv\Scripts\Activate.ps1")  
(julio) PS C:\Users\Windows 10\Desktop\julio>
```

Impletacion de modelos de machine learning

Código:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix


np.random.seed(789)

n = 500

df_emp = pd.DataFrame({
    "experiencia": np.random.normal(5,2,n),
    "certificaciones": np.random.poisson(3,n),
    "habilidades_sociales": np.random.uniform(1,10,n),
    "remoto": np.random.randint(0,2,n)
})

df_emp["salario"] = 25000 + df_emp["experiencia"]*5000 +
df_emp["certificaciones"]*2000 + np.random.normal(0,3000,n)
```

```

from scipy.special import expit

prob = expit(-2 + 0.0001*df_emp["salario"] + 0.2*df_emp["habilidades_sociales"])
df_emp["retencion"] = np.random.binomial(1, prob, n)


X = df_emp[["experiencia","certificaciones"]]
y = df_emp["salario"]
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)


modelo_reg = LinearRegression().fit(X_train,y_train)
print("Coeficientes salario:", modelo_reg.coef_)
print("Intercepto:", modelo_reg.intercept_)


X = df_emp[["experiencia","habilidades_sociales","salario"]]
y = df_emp["retencion"]
X_scaled = StandardScaler().fit_transform(X)
Xtr, Xte, ytr, yte = train_test_split(X_scaled,y,test_size=0.2,random_state=42)


knn = KNeighborsClassifier(n_neighbors=5).fit(Xtr,ytr)
pred = knn.predict(Xte)
print("\nPrecisión KNN:", accuracy_score(yte,pred))


X_clust = StandardScaler().fit_transform(df_emp[["experiencia","salario"]])
kmeans = KMeans(n_clusters=3, random_state=42).fit(X_clust)
df_emp["cluster"] = kmeans.labels_


plt.figure(figsize=(8,5))

```

```
plt.show()
```

Clasificación de instrucciones

Código:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix

np.random.seed(444)
n = 400

df_red = pd.DataFrame({
    "latencia": np.random.normal(20,5,n),
    "intentos_fallidos": np.random.poisson(1,n),
    "tamano_paquete": np.random.normal(500,100,n)
})

df_red["es_ataque"] = np.where((df_red["intentos_fallidos"]>2) |
(df_red["latencia"]>35), 1, 0)

X = df_red.drop("es_ataque", axis=1)
y = df_red["es_ataque"]
```

```
Xtr, Xte, ytr, yte = train_test_split(X,y,test_size=0.2,random_state=42)
```

```
logreg = LogisticRegression().fit(Xtr,ytr)
```

```
pred_log = logreg.predict(Xte)
```

```
print("Precisión Logística:", accuracy_score(yte,pred_log))
```

```
Xs = StandardScaler().fit_transform(X)
```

```
Xtr_s, Xte_s, ytr, yte = train_test_split(Xs,y,test_size=0.2,random_state=42)
```

```
knn = KNeighborsClassifier(n_neighbors=3).fit(Xtr_s,ytr)
```

```
pred_knn = knn.predict(Xte_s)
```

```
print("Precisión KNN:", accuracy_score(yte,pred_knn))
```

```
clust = KMeans(n_clusters=2, random_state=42).fit_predict(Xs)
```

```
df_red["cluster"] = clust
```

```
print("\nComparación Clusters vs Realidad:\n", pd.crosstab(df_red["es_ataque"],  
df_red["cluster"]))
```

```
PS C:\Users\Windows 10\Desktop\julio> & 'c:\Users\Windows 10\Desktop\julio\.venv\Scripts\python.exe' 'c:\Users\Windows 10\.vscode\extension  
s\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55481' '--' 'C:\Users\Windows 10\Desktop\julio\clasificacion de intru  
siones.py'  
Precisión Logística: 1.0  
Precisión KNN: 1.0  
  
Comparación Clusters vs Realidad:  
cluster    0    1  
es_ataque  
0         170  199  
1          24    7
```